

Lecture 12

Fundamentals of Numerical Analysis

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

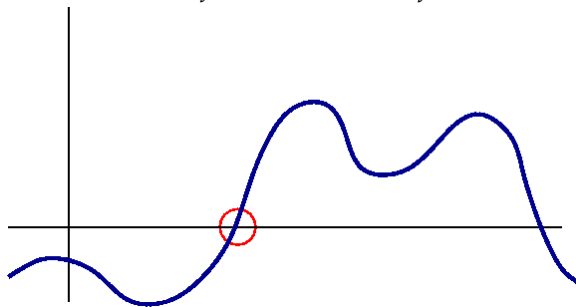
What Next?

We study iterative methods, moving from simple to more complex settings:

- **1-D root finding:** Solve a single equation $f(x) = 0$ by fixed-point or Newton iterations.
- **Systems of equations:** Extend to vector problems $F(x) = 0$ or linear systems $Ax = b$.
- **Ordinary differential equations (ODEs):** Use iterative time-stepping schemes to advance solutions.
- ...

Root Finding

Given a function $f(x)$, find x so that $f(x) = 0$



Rootfinding

Goals:

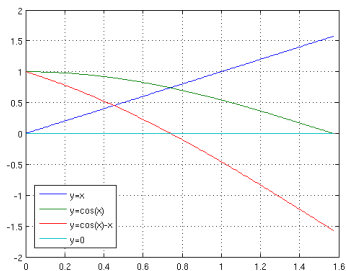
- Find roots to equations
 - Compare usability of different methods
 - Compare convergence properties of different methods
- 1 bracketing methods
 - 2 Bisection Method
 - 3 Newton's Method
 - 4 Secant Method
 - 5 (opt) fixed point iterations
 - 6 (opt) special Case: Roots of Polynomials

Roots of $f(x)$

- Any single valued function can be written as $f(x) = 0$

Example

- Find x so that $\cos(x) = x$
- That is, find where $f(x) = \cos(x) - x = 0$



Analyze your Application

- Is the function complicated to evaluate?
 - lots of expressions?
 - singularities?
 - simplify? polynomial?
- How accurate does our root need to be?
- How fast/robust should our method be?

!

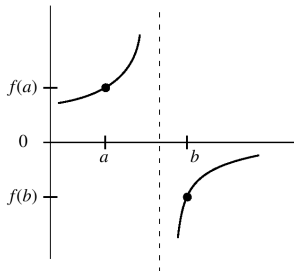
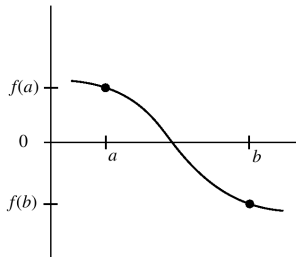
From this, you can pick the right method...

Basic Root Finding Strategy

- 1 Plot the function
 - Get an initial guess
 - Identify problematic parts
- 2 Start with the initial guess and iterate

Bracket Basics

- A root x is *bracketed* on $[a, b]$ if $f(a)$ and $f(b)$ have opposite sign.
- Changing signs does not guarantee bracketed, however: singularity



- Bracketing helps get an initial guess

Bracket Algorithm

Listing 1: Bracket Algorithm

```
1 given:  $f(x)$ ,  $x_{min}$ ,  $x_{max}$ ,  $n$ 
2
3  $dx = (x_{max} - x_{min})/n$ 
4  $x_{left} = x_{min}$ 
5  $i = 0$ 
6
7 while  $i < n$ 
8    $i = i + 1$ 
9    $x_{right} = x_{left} + dx$ 
10  if  $f(x)$  changes sign in  $[x_{left}, x_{right}]$ 
11    save  $[x_{left}, x_{right}]$  as an interval with a root
12  end
13   $x_{left} = x_{right}$ 
```

Testing Sign

$$f(a) \times f(b) < 0$$

Should we use?

```
fa = myfunc(a);  
fb = myfunc(b);
```

```
if(fa*fb<0)  
    (save)  
end
```

Better Sign Test

!

Nope. Underflow...

`sign()`

Use matlab's sign

```
fa = myfunc(a);
```

```
fb = myfunc(b);
```

```
if(sign(fa) ~= sign(fb))
```

```
    (save)
```

```
end
```

Moving forward...

Bracketing is fine. But we need to find the actual root:

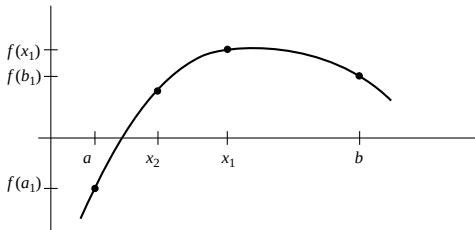
- Bisection
- Newton's Method
- Secant Method
- Fixed Point Iteration

Process:

- 1 search brackets with these methods

Bisection

Given a bracketed root, halve the interval while continuing to bracket the root



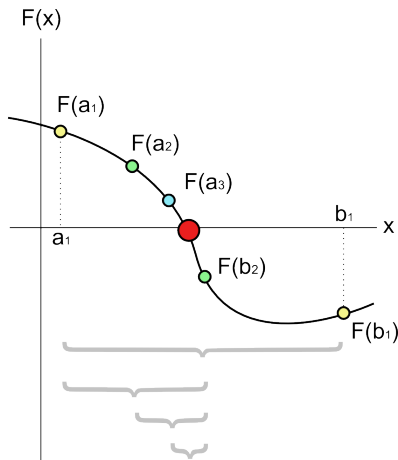
Bisection (2)

For the bracket interval $[a, b]$ the midpoint is

$$x_m = \frac{1}{2}(a + b)$$

idea:

- 1 split bracket in half
- 2 select the bracket that has the root
- 3 goto step 1



Bisection Algorithm

Listing 2: Bisection

```
1 initialize:  $a = \dots, b = \dots$ 
2 for  $k = 1, 2, \dots$ 
3    $x_m = a + (b - a)/2$ 
4   if  $\text{sign}(f(x_m)) = \text{sign}(f(x_a))$ 
5      $a = x_m$ 
6   else
7      $b = x_m$ 
8   end
9   if converged, stop
10 end
```

Bisection Example

Solve with bisection:

$$x - x^{1/3} - 2 = 0$$

k	a	b	x_{mid}	$f(x_{mid})$
0	3	4		
1	3	4	3.5	-0.01829449
2	3.5	4	3.75	0.19638375
3	3.5	3.75	3.625	0.08884159
4	3.5	3.625	3.5625	0.03522131
5	3.5	3.5625	3.53125	0.00845016
6	3.5	3.53125	3.515625	-0.00492550
7	3.51625	3.53125	3.5234375	0.00176150
8	3.51625	3.5234375	3.51953125	-0.00158221
9	3.51953125	3.5234375	3.52148438	0.00008959
10	3.51953125	3.52148438	3.52050781	-0.00074632

Analysis of Bisection

Let δ_n be the size of the bracketing interval at the n^{th} stage of bisection. Then

$$\delta_0 = b - a = \text{initial bracketing interval}$$

$$\delta_1 = \frac{1}{2}\delta_0$$

$$\delta_2 = \frac{1}{2}\delta_1 = \frac{1}{4}\delta_0$$

$$\vdots$$

$$\delta_n = \left(\frac{1}{2}\right)^n \delta_0$$

$$\implies \frac{\delta_n}{\delta_0} = \left(\frac{1}{2}\right)^n = 2^{-n}$$

$$\text{or} \quad n = \log_2 \left(\frac{\delta_n}{\delta_0} \right)$$

Analysis of Bisection

$$\frac{\delta_n}{\delta_0} = \left(\frac{1}{2}\right)^n = 2^{-n} \quad \text{or} \quad n = \log_2 \left(\frac{\delta_n}{\delta_0}\right)$$

n	$\frac{\delta_n}{\delta_0}$	function evaluations
5	3.1×10^{-2}	7
10	9.8×10^{-4}	12
20	9.5×10^{-7}	22
30	9.3×10^{-10}	32
40	9.1×10^{-13}	42
50	8.9×10^{-16}	52

Convergence Criteria

An automatic root-finding procedure needs to monitor progress toward the root and stop when current guess is close enough to the desired root.

- Convergence checking will avoid searching to unnecessary accuracy.
- Check how closeness of successive approximations

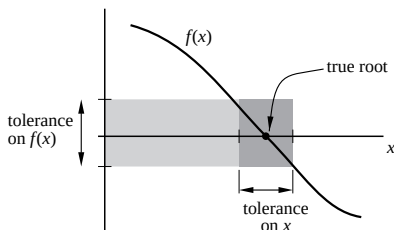
$$|x_k - x_{k-1}| < \delta_x$$

- Check how close $f(x)$ is to zero at the current guess.

$$|f(x_k)| < \delta_f$$

- Which one you use depends on the problem being solved

Convergence Criteria on x



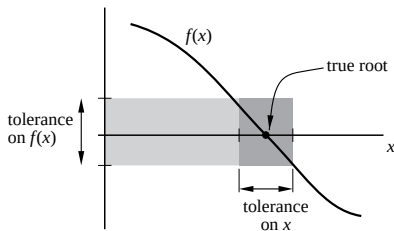
x_k = current guess at the root

x_{k-1} = previous guess at the root

Absolute tolerance: $|x_k - x_{k-1}| < \delta_x$

Relative tolerance: $\left| \frac{x_k - x_{k-1}}{b - a} \right| < \hat{\delta}_x$

Convergence Criteria on $f(x)$



Absolute tolerance: $|f(x_k)| < \delta_f$

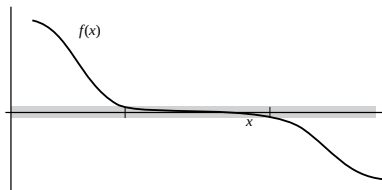
Relative tolerance:

$$|f(x_k)| < \hat{\delta}_f \max\{|f(a_0)|, |f(b_0)|\}$$

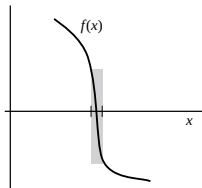
where a_0 and b_0 are the original brackets

Convergence Criteria Compared

If $f'(x)$ is small near the root, it is easy to satisfy tolerance on $f(x)$ for a large range of Δx . The tolerance on Δx is more conservative



If $f'(x)$ is large near the root, it is possible to satisfy the tolerance on Δx when $|f(x)|$ is still large. The tolerance on $f(x)$ is more conservative



Relationship Between Criteria

- How are the criteria on x and $f(x)$ related? Consider the ratio of the two criteria

$$\frac{f(x_b) - f(x_a)}{x_b - x_a}$$

- The limit of this as x_a and x_b converge to the exact answer x^* is just $f'(x^*)$.
- We can thus expect (this is not yet a proof) that

$$|f(x_b) - f(x_z)| \approx |f'(x^*)||x_b - x_a|$$

as x_a and x_b approach the solution x^* .

Convergence rate of a root finding iteration

- Let $e_n = x^* - x_n$ be the error.
- In general, a sequence is said to converge with rate r if

$$\lim_{k \rightarrow \infty} \frac{|e_{n+1}|}{|e_n|^r} = C$$

Special Cases:

- If $r = 1$ and $C = 1$, then the rate is *sublinear*
- If $r = 1$ and $C < 1$, then the rate is *linear*
- If $r > 1$ (i.e. $r = 1$ and $C = 0$), then the rate is *superlinear*
- If $r = 2$ and $C > 0$, then the rate is *quadratic*
- If $r = 3$ and $C > 0$, then the rate is *cubic*

Example

Convergence Rate

- ① $10^{-2}, 10^{-3}, 10^{-4}, 10^{-5} \dots$
- ② $10^{-2}, 10^{-4}, 10^{-6}, 10^{-8} \dots$
- ③ $10^{-2}, 10^{-3}, 10^{-5}, 10^{-8} \dots$
- ④ $10^{-2}, 10^{-4}, 10^{-8}, 10^{-16} \dots$
- ⑤ $10^{-2}, 10^{-6}, 10^{-18}, \dots$

Example

Convergence Rate

- ❶ $10^{-2}, 10^{-3}, 10^{-4}, 10^{-5} \dots$ (linear with $C = 10^{-1}$)
- ❷ $10^{-2}, 10^{-4}, 10^{-6}, 10^{-8} \dots$ (linear with $C = 10^{-2}$)
- ❸ $10^{-2}, 10^{-3}, 10^{-5}, 10^{-8} \dots$ (superlinear, not quadratic)
- ❹ $10^{-2}, 10^{-4}, 10^{-8}, 10^{-16} \dots$ (quadratic)
- ❺ $10^{-2}, 10^{-6}, 10^{-18}, \dots$ (cubic)

- Linear: Adds one digit of accuracy at each step
- Quadratic: Doubles the number of digits at each step